MODULE 21 · DOMAIN 3 - DEPLOYMENT AND ORCHESTRATION

Endpoint architecture

Multi-model, multi-container, inference pipelines, and what runs inside them.

MODULE 21 of 43 DOMAIN 3 - Deploy WEIGHT 24%

admin@sthithapragna.com

Fifty models, each called twice an hour, does not justify **fifty endpoints**.

What can sit behind a real-time endpoint

OPTION	WHAT IT HOLDS
Single-model endpoint	One model, one container. The default
Multi-model endpoint	Thousands of models sharing ONE container. Same framework
Multi-container endpoint	Up to five containers, different frameworks, invoked independently
Inference pipeline	2 to 15 containers, run in SEQUENCE for each request

ON THE EXAM

Same framework means multi-model. Different frameworks means multi-container. In sequence means a pipeline.

Multi-model endpoints

- Host **up to thousands of models** behind one endpoint, sharing a single container.
- The models must use the **same framework**, because there is only one container to run them.
- Models live in **S3 under a common prefix**. Adding one means copying a file there.
- You name the model in the invocation, using the **TargetModel** parameter.
- The classic case: one model per customer, per region, or per product line.

> ON THE EXAM
>
> Adding a model is an S3 copy, not a deployment. That alone is the reason teams adopt it.

Models are loaded on demand, not up front

- SageMaker AI does **not** download every model when the endpoint is created.
- On first invocation it **downloads that model from S3** to the instance and loads it into memory.
- It then **caches** it, so subsequent calls to the same model are fast.
- Under memory pressure it **unloads** models that have not been used recently.
- CloudWatch reports a **cache hit rate**, which tells you how well the instance is sized.

> ON THE EXAM
>
> This is why thousands of models fit: only the actively used ones occupy memory at any moment.

Auto scaling a multi-model endpoint

- **InvocationsPerInstance is a poor signal here**, because models differ in cost and cache state.
- Watch **memory utilisation** and the **cache hit rate** as well as invocation count.
- A falling cache hit rate means models are being evicted, so more memory is the fix, not more instances.
- More instances also means each instance caches a different subset, which can lower the hit rate.
- Module 23 covers auto scaling policies properly. This is the multi-model wrinkle.

ON THE EXAM

Scaling out a multi-model endpoint can make latency worse, because each new instance starts with an empty cache.

What multi-model endpoints give up

- **A model not in memory pays a download and load penalty** on first use.
- So **infrequently invoked models incur additional latency**, sometimes seconds.
- AWS is explicit: for **consistently low latency**, a traditional endpoint is still the best choice.
- Callers should retry on **ModelNotReadyException**, which appears while a model is loading.
- All models share one container's resources, so a heavy model affects its neighbours.

> ON THE EXAM
>
> The trade is explicit: large savings across many models, in exchange for variable first-call latency on each one.

Multi-model endpoints on GPU

- **GPU-backed multi-model endpoints are supported only through NVIDIA Triton Inference Server.**
- Triton supports TensorRT, PyTorch, ONNX, XGBoost, scikit-learn, Python and C++ backends.
- The **SageMaker AI Inference Toolkit is CPU-only** for multi-model endpoints.
- CPU-backed custom containers integrate the **Multi Model Server** library instead.
- Only one multi-model-enabled container may appear in a serial inference pipeline.

> ON THE EXAM
>
> If a scenario needs GPU multi-model hosting and does not mention Triton, that is the gap the answer should fill.

Multi-container endpoints

- **Up to five containers** on one endpoint, using **different models and frameworks**.
- AWS reported savings of **up to 80 percent** against running each on its own endpoint.
- Suits models with **similar resource needs** that individually lack traffic to fill an instance.
- **Per-container CloudWatch metrics**, so you can see each one separately.
- Dev and test endpoints are a common use, alongside infrequently invoked production models.

> ON THE EXAM
>
> Five is the limit. A scenario naming six or more different frameworks has ruled this option out.

Direct or serial invocation?

DIRECT MODE

Independent models

- Each container invoked on its own.
- Named with TargetContainerHostname.
- For unrelated models sharing an instance.
- IAM can restrict which are callable.

SERIAL MODE

A pipeline

- Containers run in sequence per request.
- Output of one feeds the next.
- This is what a pipeline is.
- The default execution mode.

THE TIEBREAKER

Multi-container and inference pipeline are the same feature with different execution modes. That surprises people.

Inference pipelines

- **Two to fifteen containers** run in sequence behind one endpoint, for every request.
- The classic shape: **preprocessing, then the model, then postprocessing**.
- Data passes between containers **in memory**, so the latency cost is small.
- The caller sends raw input and receives a final answer. The stages are invisible to them.
- Works for both real-time endpoints and batch transform jobs.

> ON THE EXAM
>
> The preprocessing container can be the SAME code used in training. That is worth more than the packaging convenience.

This is the skew fix, deployed

- Module 04's problem: features built one way for training and another way at inference.
- An inference pipeline lets the **same preprocessing code** run in both places.
- The fitted scaler or encoder from module 05 is packaged and deployed with the model.
- So the calling application sends raw values and does no feature engineering at all.
- The Feature Store from module 06 solves the same problem for shared, precomputed features.

ON THE EXAM

Training-serving skew has two structural fixes: one implementation in a pipeline, or one source of truth in a Feature Store.

Which hosting option

WHAT THE SCENARIO SAYS	THE OPTION
One model, normal traffic	Single-model endpoint
Many models, same framework, shared container	Multi-model endpoint
Many models, each needs consistent low latency	Separate endpoints, not multi-model
A few models, different frameworks, invoked separately	Multi-container, direct mode
Preprocessing must run with the model	Inference pipeline
Thousands of customer-specific models	Multi-model endpoint
More than five different frameworks	Not multi-container. Reconsider the design

ON THE EXAM

Row three is the one people miss. If consistent latency is the requirement, multi-model is the wrong answer.

Compatibility between options

COMBINATION	SUPPORTED?
Multi-model on real-time endpoints	Yes, CPU and GPU. GPU requires Triton
Multi-model on serverless	No
Inference pipeline on real-time	Yes
Inference pipeline on batch transform	Yes
Multi-model inside an inference pipeline	Yes, but only ONE such container
Multi-container direct mode on batch transform	No. Direct invocation is a real-time concept

ON THE EXAM

Serverless is the simple option and it stays simple. The advanced hosting patterns are real-time features.

Updating models on these endpoints

- **Multi-model:** upload a new artefact to S3. It is picked up without touching the endpoint.
- A model already cached in memory may serve the **old version** until it is evicted or reloaded.
- So version the S3 key rather than overwriting, and point callers at the new key.
- **Single-model, multi-container and pipelines** update by creating a new endpoint configuration.
- That update path is what module 24's blue/green and canary strategies operate on.

> ON THE EXAM
>
> Overwriting a multi-model artefact in place gives you a silent, partial rollout with no way to say which version served a request.

Monitoring these architectures

- **Multi-container** gives per-container CloudWatch metrics, so each model is visible separately.
- **Multi-model does not give per-model metrics.** You see the endpoint, not the individual models.
- So a single misbehaving model among two thousand is genuinely hard to find.
- MME-specific metrics do exist: cache hit rate, model loading time, models loaded.
- Data capture from module 20 still works, and records which model was invoked.

ON THE EXAM

Observability is part of the isolation trade. Utilisation goes up, and visibility into any one model goes down.

Multi-tenancy and isolation

- A multi-model endpoint puts many tenants' models in **one container's memory**.
- Isolation is by model name, enforced by whichever service decides the TargetModel value.
- **Multi-container direct mode can enforce isolation with IAM**, using condition keys on the container name.
- With multi-model, there is no equivalent IAM control per model. Your application is the boundary.
- For regulated tenants, separate endpoints remain the defensible answer.

> ON THE EXAM
>
> Module 09's rule again: isolation enforced by IAM survives a bug in your code; isolation enforced by a parameter does not.

The container contract

- A SageMaker AI endpoint is a **Docker container serving HTTP**, stored in Amazon ECR.
- It must answer **/ping** for health checks and **/invocations** for predictions.
- The model artefact is downloaded from S3 and made available at a known path.
- **AWS provides prebuilt containers** for the common frameworks, so most teams never build one.
- Building your own is out of scope in MLA-C02. Module 40 covers it for C01 candidates.

ON THE EXAM

Two HTTP endpoints and a file path. The contract is small, which is why so many frameworks can be adapted to it.

What is inside model.tar.gz

CONTENTS	NOTE
The model artefact	Whatever the framework saved: weights, a booster file, a pickle
Inference code	For script mode, the entry point that loads and predicts
requirements.txt	Extra packages, installed when the container starts
Anything else your code needs	A fitted scaler, a vocabulary, a label mapping
Where it goes	S3. SageMaker AI downloads and extracts it into the container

ON THE EXAM

The fitted transformer from module 05 travels here. If it is not in the tarball or the pipeline, it is not deployed.

ECR and image versioning

- The container image lives in **Amazon ECR**, referenced by URI in the model definition.
- **Do not reference 'latest'.** A moving tag changes your endpoint behaviour with no deployment.
- Pin the exact image URI, the same discipline as pinning a model version in module 16.
- AWS prebuilt images are versioned by framework release, so you pin the framework too.
- The execution role needs **ECR pull permission**, or the endpoint fails to start.

> ON THE EXAM
>
> The image, the artefact and the endpoint configuration together define behaviour. All three need versioning.

Inference components

- Place several models on an endpoint with **explicit resource allocation** per model.
- You specify CPU, memory and accelerators for each, rather than sharing what is available.
- Each component can be **scaled independently**, including down to zero copies.
- Aimed at packing large models efficiently onto expensive GPU instances.
- A more controlled alternative to multi-model for a smaller number of larger models.

> ON THE EXAM
>
> Multi-model endpoints share resources implicitly. Inference components allocate them explicitly, which matters most on GPU.

Hosting a model outside SageMaker AI

TARGET	WHEN IT MAKES SENSE
Amazon ECS or EKS	The organisation already runs containers there, with its own tooling
AWS Lambda	A tiny model, low volume, and no endpoint wanted. Module 20
EC2 directly	Full control, and full responsibility. Rarely the exam's answer
IoT Greengrass	Inference at the edge, on a device with intermittent connectivity
SageMaker AI endpoint	The default, and what the exam expects unless told otherwise

ON THE EXAM

Existing platform investment is the usual reason to leave. It is legitimate, and it should be stated in the scenario.

Deploying from the Model Registry

- A registered **model package** holds the artefact, the image, metrics and approval status.
- **Deploy from an approved version**, so nothing untested reaches an endpoint.
- Approval status is what a pipeline gates on, following module 17's ConditionStep.
- The registry records which version is deployed where, which an audit will ask for.
- Module 28 automates this. Here it is the link between training and hosting.

ON THE EXAM

Deploying an artefact straight from a training job skips the gate. The registry exists so that cannot happen by accident.

What each option costs

OPTION	COST SHAPE
Single-model endpoint	One instance bill per model. Scales linearly with model count
Multi-model endpoint	One instance bill for thousands. The largest saving available
Multi-container endpoint	One instance bill for up to five. Up to 80 percent saving
Inference pipeline	One instance bill. Extra containers add memory need, not a charge
Inference components	One instance bill, allocated explicitly. Idle components scale to zero

ON THE EXAM

All of these save money the same way: by raising utilisation of an instance you are already paying for.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Occasional multi-second latency, multi-model endpoint	Cold model load. Expected behaviour
ModelNotReadyException returned to callers	Model still loading. Retry
Low cache hit rate on a multi-model endpoint	Instance too small for the access pattern
Cannot deploy a sixth container	The multi-container limit is five
GPU multi-model endpoint will not start	Not using the Triton Inference Server
Predictions differ between training and serving	Preprocessing not in the pipeline. Module 04
One model slows all the others	Shared container resources on a multi-model endpoint

ON THE EXAM

Rows one and two are documented behaviour, not faults. Knowing that saves a long investigation.

These options trade **isolation for utilisation**. Know which one the scenario is paying for.

A SaaS company trains one XGBoost forecasting model per customer, and has around 2,400 of them. All use the same framework and container image.

Any customer's model may be called at any time, but most are called only a few times a day. A one to two second delay on the first call of the day is acceptable. Cost is the stated priority.

How should they host these models?

A 2,400 single-model endpoints, one per customer, with auto scaling

B A multi-model endpoint, with all model artefacts under one S3 prefix

C 480 multi-container endpoints, five customers on each

D Batch transform, run hourly across all customers

◆ PAUSE AND DECIDE

How should they host these models?

A 2,400 single-model endpoints, one per customer, with auto scaling

Perfect isolation, and 2,400 instance bills for models called a few times a day. It is precisely the problem multi-model endpoints exist to solve.

B A multi-model endpoint, with all model artefacts under one S3 prefix

Same framework, same container, thousands of models, sparse access, and a stated tolerance for first-call latency. Every condition matches.

C 480 multi-container endpoints, five customers on each

Multi-container is for different frameworks. These share one, and 480 endpoints is still an enormous amount of billing and management.

D Batch transform, run hourly across all customers

Any model may be called at any time, so a caller is waiting. Hourly batch cannot answer an on-demand request.

THE SIGNAL

The sentence about tolerating one to two seconds is the permission slip. Without it, cold model loads would be the objection.

A team has four models used by different parts of an application: a TensorFlow image classifier, a PyTorch text model, a scikit-learn scorer and an XGBoost ranker.

Each is invoked independently, none has enough traffic to fill an instance, and all have similar CPU and memory needs. They want per-model metrics, and to restrict which teams can call which model.

What should they deploy?

A. A multi-model endpoint with all four artefacts under one prefix

B. An inference pipeline chaining all four containers in sequence

C. Four single-model endpoints, one per framework

D. A multi-container endpoint in direct mode, with IAM conditions on TargetContainerHostname

◆ PAUSE AND DECIDE

What should they deploy?

A. A multi-model endpoint with all four artefacts under one prefix

Multi-model shares ONE container, so every model must use the same framework. Four frameworks cannot share one runtime.

B. An inference pipeline chaining all four containers in sequence

Serial execution means every request passes through all four. These models are invoked independently for different purposes.

C. Four single-model endpoints, one per framework

Works, and wastes four instances on models that individually cannot fill one. The scenario says none has enough traffic.

D. A multi-container endpoint in direct mode, with IAM conditions on TargetContainerHostname

Four frameworks, invoked independently, similar resource needs, under the five-container limit. Direct mode gives per-container metrics and IAM isolation.

THE SIGNAL

Four frameworks rules out multi-model. Invoked independently rules out serial. Four is under the limit of five.

A credit team's model requires scaling and one-hot encoding before inference. Today the calling application performs those steps in Java, using logic written to match the scikit-learn transformers used in training.

Production accuracy is below validation accuracy, and an audit finds the two implementations handle unseen categories differently.

What should they deploy?

A. An inference pipeline: the fitted scikit-learn transformer container, then the model container

B. A multi-model endpoint holding both the transformer and the model

C. A multi-container endpoint in direct mode, with the application calling each in turn

D. Keep the Java preprocessing and add unit tests asserting it matches scikit-learn

◆ PAUSE AND DECIDE

What should they deploy?

A An inference pipeline: the fitted scikit-learn transformer container, then the model container

The exact transformer fitted during training is deployed with the model. One implementation, so there is nothing left to diverge.

B A multi-model endpoint holding both the transformer and the model

Multi-model serves one named model per request. It does not chain them, so the transformer output would never reach the model.

C A multi-container endpoint in direct mode, with the application calling each in turn

Right containers, wrong mode. Direct means the application orchestrates two calls; serial mode is the pipeline that does it for you.

D Keep the Java preprocessing and add unit tests asserting it matches scikit-learn

Two implementations survive, policed by tests. Module 04's answer: that reduces the odds without removing the cause.

THE SIGNAL

Note that C is nearly right. The containers are correct and the execution mode is wrong, which is exactly what the two modes are for.

A trading platform runs three pricing models, each on its own endpoint. Every request must complete within 15 milliseconds at the 99th percentile, and all three are called continuously throughout the trading day.

To reduce cost, an engineer proposes consolidating them onto a single multi-model endpoint, noting that all three use XGBoost.

How should you respond?

A Agree. Same framework means multi-model is the correct consolidation

B Agree, but use a multi-container endpoint instead for per-model metrics

C Reject it. Multi-model trades consistent latency for utilisation, and latency is the requirement

D Reject it. Multi-model endpoints do not support XGBoost

◆ PAUSE AND DECIDE

How should you respond?

A Agree. Same framework means multi-model is the correct consolidation

Applies the framework rule and ignores the latency requirement. AWS states plainly that consistently low latency calls for a traditional endpoint.

B Agree, but use a multi-container endpoint instead for per-model metrics

Three containers still share one instance's resources, competing for CPU on every request. It does not protect a 15 ms p99 either.

C Reject it. Multi-model trades consistent latency for utilisation, and latency is the requirement

Three continuously-called models cannot generate much saving anyway, and shared resources plus cold loads threaten the p99 target.

D Reject it. Multi-model endpoints do not support XGBoost

Factually wrong. XGBoost is supported, including on GPU through Triton. Right conclusion, invented reason.

THE SIGNAL

The framework rule says multi-model is possible. The latency requirement says it is unwise. Possible and correct are different.

What to take from this module

- **Same framework, multi-model. Different frameworks, multi-container. In sequence, pipeline.**
- **Multi-model loads models on demand**, so first-call latency is variable. Not for strict SLAs.
- **Multi-container is capped at five**, and has direct and serial execution modes.
- **Inference pipelines deploy preprocessing with the model**, which fixes skew structurally.
- **These trade isolation for utilisation.** Read which one the scenario is paying for.

ON THE EXAM

Module 22 covers what the endpoint runs on: instance families, Inferentia, Trainium, and inference optimisation.